To-Do's

- minimale dominanz und prominenz konsistent machen, statt min prominenz und dominanz hioer Dominanzgrenzwert verwenden . . . 2
- stimmt der letzte satz und soll ich ihn umsetzen? . . . 2
- Testgruppen erklären als gleiche tests mit unterschiedlichen Vergleichsbereichen 3
- verweis auf formeln Precision und Recall . . . 4
- maybe in Theorieteil . . . 5
- neu formulieren den satz . . . 5
- dabei austauschen . . . 5
- besserer Satzbnau (verstehe ihn selbst nicht mal richtig) . . . 6
- sicherstellen, dass das in NNeues Testkonzeptßteht . . . 6
- Diagramm uml of test_controler . . . 6
- ist das wirklich so? . . . 7
- Diagramm uml of test_runner . . . 8
- ist das wirklich so? . . . 8
- maybe hier diagramm über genaue ausführreichenfolge und datenübergabelogik und dependencies? . . . 9
- bild Wetterstein . . . 9
- umformulieren . . . 10
- bild Feldberg Kartenausschnitt . . . 10
- bild Südschwarzwald . . . 10
- zitat zu Tester-Studienarbeit . . . 10
- abkürzung . . . 11
- border with auf deutsch formulieren . . . 13
- verweis auf unittests . . . 13
- abgleich wirklich geleich emtriken wie in 4.2 . . . 13
- diese überleitung umformulieren . . . 13
- matching areas auf pixelwidth/2 bei pixelgenauc und rest auf 100m und 1000m circular setzen . . . 13

problem solstein 13
schlussfoldern, das berechnung der Distanz Müll 15
diagram mit Zylinder oder berg und dann ne border durch/ höhenlinien an border 16
das border width problem diagramm 16
geringe Dominanz sowie Prominenzabweichungen 18
Laufzeittests in 5 zu finden 18
besserer Name für die datei und konsistenz dieses namens 20
intro zu 5 21
achtung kann wie ki klingen 21
ref def Prominenz 22
falls in ui verwendet: Sicherheitswert in Prozent der Berechnugn der Dominaz und Prominenz 26
performance optimierungen in eigenenm section (die size wurde erhöht bei maximumfilter) 28
tests interpretieren und bewerten. erst ganz am ende von der PA wenn sich nichts mehr ändert 28
vereinheitlichen über Kapitel hinweg die bezeichnung der einzeldateianalyse Klasse 34
filemap erstellen 34
Dateien positionen bestimmen 34
globale Randgipfel ausschließen 34
Prominenzberechnung über die Dateigrenzen hinweg 34
Dominanzberechnung über die Dateigrenzen hinweg 34
vorschalg aus alter pa in text einarbeiten 34
link zu ui reaktivität 35
intro zu 7 37
Abkürzung UI in acronyms.tex 37
rechtschreibung hier? 39
- multi threading kontextwechsel langsam - daten je Prozess - auf Multithreading ändern - Fortschrittanzeige die Gewichte besser amchen oder dynamischer - Ausblick der raster search Algorithmus für Dominanz 39

Overkomplex Title

T2000

des Studiengangs „Informatik“

an der Dualen Hochschule Baden-Württemberg, Stuttgart

von

Max Musterman

Abgabedatum

8. September 2025

Bearbeitungszeitraum	3 Monate
Matrikelnummer	5321326
Kurs	INF23A
Ausbildungsfirma	TRUMPF SE + Co. KG, Ditzingen
Betreuer	Betreuer
Zweitbetreuer	

Erklärung

Ich versichere hiermit, dass ich die vorliegende Arbeit mit dem Titel „Overkomplex Title“ selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet habe.

Ort	Datum	Unterschrift
		(Max Musterman)

Inhaltsverzeichnis

Abkürzungsverzeichnis **III**

Abbildungsverzeichnis **IV**

Quellcodeverzeichnis **V**

0.1 Aktuelles Testkonzept 1

1 Bewertung des Gipfelfindungsalgorithmus **2**

1.1 Motivation 2

1.2 Neues Testkonzept 2

1.3 Umsetzung 6

1.3.1 Test_Controler 6

1.3.2 Test_Runner 8

1.4 Testen des Algorithmus 9

1.4.1 Allgemeine Bewertung 9

1.4.2 Definition der Test-Szenarien 9

1.4.3 Durchlauf der Tests 11

1.4.4 Beschaffung der Testdaten 11

1.4.5 Bewertung der Testergebnisse 13

1.5 Bewertung des Testers 18

2 Optimierung des Algorithmus **21**

2.1 Motivation 21

2.2 Vorgehen 21

2.3 Verbesserung der Abbruchbedingung und -logik 22

2.4 Verbesserung der Distanzberechnung 23

2.5 Neubewertung des Randwertproblems 25

2.6 Verbesserung der Maximabestimmung 28

2.7 Einordnung der Laufzeitverbesserungen 28

2.8 Bewertung 30

3 Verarbeitung mehrerer GeoTIFF-Dateien 32

3.1 Motivation 32

3.2 Ausführung mehrerer Dateien 33

3.3 Zusammenführung der Ergebnisse 34

3.4 Parallelisierung 34

4 Reengineering der Benutzeroberfläche 37

4.1 Motivation 37

4.2 Umsetzung 38

4.2.1 Fortschritt-Visualisierung 38

Abkürzungsverzeichnis

Abbildungsverzeichnis

1.1 Dominanz von Daniel, Birkkarspitze, Hochplattig und Große Arnspitze 16

2.1 Satelitenbild des Zugspitzecks 23

2.2 2D-Geländeprofil zur Illustration von Prominenz und Dominanz eines Gipfels 26

2.3 2D-Geländeprofil zur Illustration der minimalen Prominenz und Dominanz bei begrenztem Kartenausschnitt. 27

2.4 2D-Geländeprofil zur Illustration der minimalen Prominenz und Dominanz bei begrenztem Kartenausschnitt. 28

2.5 2D-Geländeprofil zur Illustration der minimalen Prominenz und Dominanz bei begrenztem Kartenausschnitt. 30

Quellcodeverzeichnis

Dies ist ein Test [**mustermann2024**].

- mulitthreading - multi prozessing - multi prozessing mit shared memory

0.1 Aktuelles Testkonzept

1 Bewertung des Gipfelfindungsalgorithmus

minimale dominanz und prominenz konsistent machen, statt min prominenz und dominanz hioer Dominanzgrenzwert verwenden

Dieses Kapitel beschreibt die Konzeption und Implementierung einer automatisierten Testumgebung für den Gipfelfindungsalgorithmus. Nach der Erläuterung des Testansatzes und dessen technischer Realisierung werden spezifische Testszenarien definiert und durchgeführt. Abschließend erfolgt eine Bewertung des Algorithmus auf Basis der ermittelten Testergebnisse.

stimmt der letzte satz und soll ich ihn umsetzen?

1.1 Motivation

Damit die Optimierungen des Gipfelfindungsalgorithmus tatsächlich zu einer Verbesserung der Laufzeit und Genauigkeit führen, muss zuerst einmal eine Möglichkeit zur Bewertung des Algorithmus geschaffen werden. Es muss eine Methode gefunden werden, um die Ergebnisse des Algorithmus mit den tatsächlichen Gipfeln zu vergleichen und die Genauigkeit zu messen. Außerdem muss eine Möglichkeit gefunden werden, um die Laufzeit des Algorithmus zu messen und zu vergleichen. Dabei ist es wichtig, dass die Bewertung möglichst automatisiert abläuft, um eine große Anzahl von Testfällen schnell und effizient bewerten zu können.

1.2 Neues Testkonzept

Im Gegensatz zum alten Testkonzept, bei dem die Ergebnisse des Algorithmus manuell mit den tatsächlichen Gipfeln verglichen wurden, soll nun ein neues Testkonzept entwickelt werden, das eine weitgehend automatisierte Bewertung des Algorithmus ermöglicht. Das Konzept umfasst die automatisierte Ausführung des Algorithmus mit vordefinierten Parametern, die automatisierte Erfassung der Ergebnisse und deren systematischen Vergleich mit den tatsächlichen Gipfeln sowie die automatisierte Berechnung von Metriken zur Bewertung der Genauigkeit und der Laufzeit.

Zu diesem Zweck soll ein Python Skript geschrieben werden, dass die obendefinierten Schritte durchführt. Das Skript soll so gestaltet sein, dass es einfach zu bedienen ist und

eine große Anzahl von Testfällen schnell und effizient definiert werden können. Ein testfall ist eine testweise Ausführung des Algorithmus auf einem Kartenstück mit vordefinierten Parametern. Dies soll mit der Hilfe einer Konfigurationsdatei erreicht werden, in der die verschiedenen Testfälle definiert werden können. In dieser Datei soll auch definiert werden, welche Testfälle bei der Ausführung des Testers gestartet werden sollen.

Die Definition eines Testfalls umfasst die Parameter *Name*, *Input-Datei*, *Output-Datei*, *Geodaten-Datei*, *Algorithmus-Parameter* sowie den *Vergleichsbereich*. Der Name dient als eindeutiger Identifier des Testfalls. Dadurch kann der Testfall nachdem er definiert wurde mit einer weiteren Funktion gestartet werdenDie Input-Datei enthält reale Gipfeldaten der Testregion. Diese umfassen für jeden Gipfel den Namen, die Koordinaten, die Höhe, die Prominenz sowie die Dominanz. Diese Daten dienen als Ground Truth zur Bewertung des Algorithmus. Die Output-Datei legt fest, in welche Datei die Testergebnisse geschrieben werden. Die Geodaten-Datei enthält die Kartendaten der Testregion im GeoTIFF-Format. Diese Daten werden dem Algorithmus als Eingabe übergeben. Die Algorithmus-Parameter definieren die Bedingungen für die Ausführung. Dazu gehören die minimale Dominanz, die minimale Prominenz, die minimale Höhe, die minimale orographische Dominanz sowie die Randbreite. Diese Parameter beeinflussen, welche Erhebungen als Gipfel klassifiziert werden. Der Vergleichsbereich definiert einen räumlichen Umkreis um jeden realen Gipfel. Innerhalb dieses Bereichs werden vom Algorithmus erkannte Gipfel einem tatsächlichen Gipfel zugeordnet.

Testfälle können auch gebündelt als Gruppe initialisiert sowie ausgeführt werden. Hierfür kann beim Erstellen jedem Testfall ein Gruppenname hinzugefügt werden. Ganze Testfallgruppen lassen sich anschließend über eine eigene Funktion definieren. Diese Funktion verwendet außer des Testgruppennamen die gleichen Parameter wie die Testfall Generierung.

Testgruppen erklären als gleiche tests mit unterschiedlichen Vergleichsbereichen

Der Testablauf besteht aus zwei Schritten. Zunächst wird der Algorithmus mit den definierten Parametern auf den Geodaten ausgeführt. Anschließend werden die Ergebnisse validiert. Zur Validierung wird das Problem der Gipfelbestimmung als Klassifikationsproblem formuliert. Der Algorithmus bestimmt alle Gipfel innerhalb des Kartenausschnitts und kann daher als binärer Klassifikator betrachtet werden. Er teilt alle Punkte in zwei

Klassen ein: *Gipfel* (positiv) und *kein Gipfel* (negativ).

Zur Bewertung eines Klassifikators wird eine Konfusionsmatrix verwendet. Eine Konfusionsmatrix stellt die Anzahl korrekt und falsch klassifizierter Fälle gegenüber. Sie besteht aus vier Kategorien: Wahre Positive (True Positives), Falsche Positive (False Positives), Wahre Negative (True Negatives) und Falsche Negative (False Negatives).

Für die Analyse mittels Konfusionsmatrix werden ausschließlich diejenigen Punkte betrachtet, die entweder in der Input-Datei als reale Gipfel definiert sind oder vom Algorithmus als Gipfel ausgegeben werden. Es werden somit nicht alle einzelnen Pixel bzw. Geländepunkte der GeoTIFF-Datei als Klassifikationsobjekte herangezogen. Stattdessen beschränkt sich die Bewertung auf die Menge der tatsächlich existierenden Gipfel (Ground Truth) sowie auf die vom Algorithmus detektierten Gipfel. Dadurch wird die Analyse auf die für die Aufgabenstellung relevanten diskreten Objekte fokussiert und eine Verzerrung durch die sehr große Anzahl an Nicht-Gipfel-Pixeln vermieden. Zudem sind Puntke die keine Gipfel sind und vom Algorithmus nicht als solche erkannt werden für die aus den Konfusionsmatrix bestimmten Metriken nicht relevant ().

verweis auf formeln Precision und Recall

Die korrekte Klassifikation ergibt sich aus der Input-Datei, in der alle realen Gipfel des Kartenausschnitts definiert sind. Da die Kartendaten pixelbasiert sind und somit eine Approximation der realen Welt darstellen, können erkannte Gipfel nicht exakt auf den realen Koordinaten liegen. Daher wird der Vergleichsbereich eingeführt. Befindet sich ein vom Algorithmus erkannter Gipfel innerhalb dieses Bereichs um einen realen Gipfel, wird er diesem zugeordnet.

Für jeden realen Gipfel darf höchstens ein erkannter Gipfel zugeordnet werden. Innerhalb eines Vergleichsbereichs darf daher kein zweiter Gipfel liegen. Um dies sicherzustellen, muss die minimale Dominanz größer sein als der Vergleichsbereich oder der Kartenausschnitt so gewählt werden, dass ein solcher Fall ausgeschlossen ist. In diesem Testsystem wird der Vergleichsbereich stets kleiner als die minimale Dominanz gewählt.

Auf dieser Grundlage gelten die in der Input-Datei definierten Gipfel als tatsächliche positive Fälle. Alle nicht in der Input-Datei definierten Punkte gelten als tatsächliche negative Fälle. Die vom Algorithmus erkannten Gipfel bilden die vorhergesagten positiven

Fälle. Nicht erkannte Punkte bilden die vorhergesagten negativen Fälle.

Im Folgenden wäre hier ein Beispiel für einen Solchen Testfall im Wetterstein Gebirge.

	Vorhergesagt: Gipfel	**Vorhergesagt: Kein Gipfel**
Tatsächlich: Gipfel	18	4
Tatsächlich: Kein Gipfel	10	0

In diesem Beispiel wurden 18 reale Gipfel korrekt erkannt (Wahre Positive). Diese Gipfel sind in der Input-Datei und der Vorhersage des Algorithmus enthalten. Vier Gipfel aus der Input-Datei waren nicht in der Ausgabe des Algorithmus enhalten, was bedeutet, dass der Algoprithmus diese nicht erkannt hat (Falsche Negative). Zehn Gipfel wurden fälschlicherweise als Gipfel klassifiziert (Falsche Positive). Null Gipfel wurden von Algorithmus sowie Input-Datei nicht als Gipfel klassifiziert, da diese immer ignoriert werden.

Auf Basis dieser Matrix können verschiedene Qualitätsmaße berechnet werden. Bei diesem Tester werden die Präzision (Precision) und die Sensitivität (Recall), sowie der F-Score (f1-Score) verwendet. Die können mit den folgenden Formeln berechnet werden:

$$\text{Precision} = \frac{\text{True Positiv (TP)}}{\text{True Positiv (TP)} + \text{False Positiv (FP)}} \tag{1.1}$$

$$\text{Recall} = \frac{\text{True Positiv (TP)}}{\text{True Positiv (TP)} + \text{False Negativ (FN)}} \tag{1.2}$$

$$\text{F1-Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{1.3}$$

maybe in Theorieteil

Diese Metriken geben Aufschluss über die Genauigkeit des Algorithmus und zeigen wo der Algorithmus genauer ist und wo schlechter . Dabei zeigt die Precision wie sehr der Algorithmus zum finden neuer nicht echter Gipfel neigt und der Recall zeigt wie häufig der Algorithmus reale Gipfel nicht erkennt. Beide Werte sind ≥ 0 und ≤ 1. Der F-Score verbindet beide Werte und zeigt wie genau der Algorithmus allgemein ist.

neu formulieren den satz

Dabei gilt desto näher die Werte an eins sind, desto besser ist der Algorithmus. Am Wich-

dabei austauschen

tigsten ist, dass der Recall möglichst nahe an der eins ist, da es für einen Suchalgorithmus besser ist unötige Werte zu finden als Werte zu übersehen.

Andere Metriken zur Bestimmung der Genauigkeit sind die durchschittlichen Abweichungen der Position, Höhe, Prominenz und Dominanz pro Testfall. Diese Werte zeigen die Genauigkeit des Algorithmus für die vom Algorithmus gefundenen realen Gipfel. Sie werden in Metern angegeben. Zusätzlich wird bei der Positionsabweichung eine Abweichung in X- und Y-Richtung bestimmt. Alle Metriken werrden in die im Testfall definierte Output-Datei geschrieben. Abgesehen von diesen Metriken werden zudem die Zuordnungen der Gipfel und die Einzelabweichungen pro Gipfel in die Output-Datei geschrieben.

Außerdem sollen Metriken implementiert werden, die die Laufzeit des Algorithmus aufzeigen und verdeutlichen, wo der Algorithmus längere Zeit benötigt als erwartet. Für diesen Zweck soll eine Funktion implementiert werden, die die gesammte Ausführtzeit sowie die einzelnen Ausführzeiten der Teilbereiche (globale Maximasuche, Prominenzberechnung, Dominanzberechnung) zurückgibt.

Zusammengefasst kann gesagt werden, dass das neue Testkonzept, bis auf die Testfalldefinition, automatisiert abläuft und dadurch, dass die Suche als Klassifikationsproblem neu formuliert wird die Genauigkeit des Algorithmus bestimmt werden kann. Außerdem liefert das neue Testkonzept eine zeitliche Abschätzung der Leitung der einzelnen Teiile des Algorithmus.

besserer Satzbnau (verstehe ihn selbst nicht mal richtig)

1.3 Umsetzung

Wie bereits in 1.2 definiert wird der Tester mit einer Konfigdatei und mehreren Pythondateien umgesetzt. Zusätzlich werden die Realen Gipfel für jeden Testfall in einer seperaten Json-Datei definiert und die Ergebnisse in selbigen Format gespeichert.

sicherstellen, dass das in NNeues Testkonzeptßteht

Der Tester besteht aus zwei Klassen. Sie heißen Test_Runner und Test_Controler.

1.3.1 Test_Controler

Diagramm uml of test_controler

Die Klasse `Test_Controler` ist die zentrale Steuerungseinheit für die automatisierten

Tests. Sie ist dafür verantwortlich, Testkonfigurationen zu definieren, zu verwalten und auszuführen.

Testdefinition Die Methode `define_test()` ermöglicht die Definition eines einzelnen Testlaufs. Dabei werden Parameter wie der Name des Tests, die zu verwendenden Geodaten, die Ausgabedatei für die Ergebnisse sowie die Schwellenwerte für die Gipfelerkennung (Prominenz, Dominanz, etc.) festgelegt. Diese Konfiguration wird als `Test_Config`-Objekt gespeichert.

Für die Durchführung von Testreihen existiert die Methode `define_test_group()`. Diese Methode generiert eine Gruppe von Tests, die sich in der Regel nur durch einen variierenden Parameter unterscheiden, wie beispielsweise dem *Matching-Radius*. Dies vereinfacht das Testen von verschiedenen Konfigurationen für ein bestimmtes Szenario.

Testausführung

Die Ausführung der Tests wird über die Methoden `run_test()` und `run_test_group()` gesteuert. `run_test()` kann entweder einen einzelnen, spezifischen Test anhand seines Namens oder alle definierten Tests ausführen. `run_test_group()` führt alle Tests aus, die zu einer bestimmten Gruppe gehören.

Intern verwaltet der `Test_Controler` eine Liste von `Test_Runner`-Instanzen. Für jede Testkonfiguration wird geprüft, ob bereits ein kompatibler `Test_Runner` (für die gleiche Geodaten-Datei oder den gleichen Ordner) existiert. Ist dies der Fall, wird dieser wiederverwendet, um das wiederholte Laden und Vorverarbeiten der Geodaten zu vermeiden. Andernfalls wird eine neue `Test_Runner`-Instanz erzeugt.

ist das wirklich so?

Zeitmessung und Parametertests

Zusätzlich zu den funktionalen Tests bietet der `Test_Controler` Methoden zur Zeitmessung (`timetest_single_file()` und `timetest_multiple_file()`). Diese führen die Gipfelanalyse für eine einzelne Datei oder mehrere Dateien durch und messen die benötigte Zeit für die einzelnen Teilschritte des Algorithmus, wie die Suche nach lokalen Maxima, die Prominenz- und die Dominanzberechnung. Die Methode `multiple_execution_test()`

ermöglicht es, die Analyse mehrfach hintereinander auszuführen, um die Stabilität und Konsistenz der Laufzeit zu bewerten. Diese Logik wurde hier im `Test_Controler` implementiert, da diese Tests wenig Programmcode zur Ausführung benötigen und eine andere Ausführlogik sowie Metriken wie die Genauigkeitstests besitzen.

Diagramm uml of test_runner

1.3.2 Test_Runner

Der `Test_Runner` ist für die Durchführung und Auswertung der Genauigkeitstests zuständig. Jede Instanz dieser Klasse ist an ein bestimmtes Set von Geodaten gebunden, welche sie bei der Initialisierung lädt und für alle nachfolgenden Tests wiederverwendet.

Initialisierung und Datenvorbereitung

Beim Erstellen eines `Test_Runner`-Objekts werden die digitalen Höhenmodelldaten (DEM) eingelesen und vorverarbeitet. Wichtige Metadaten wie das Koordinatenreferenzsystem, die Auflösung und die Umrechnungsfaktoren zwischen Pixel und Meter werden extrahiert und für spätere Berechnungen zwischengespeichert. Dieser Schritt ist ressourcenintensiv, weshalb der `Test_Controler` darauf achtet, `Test_Runner`-Instanzen für bereits geladene Geodaten wiederzuverwenden.

ist das wirklich so?

Testdurchführung

Die zentrale Methode ist `execute_test()`, die eine spezifische Testkonfiguration entgegennimmt. Der Ablauf eines Tests ist wie folgt:

1. **Einlesen der Testdaten:** Die realen Gipfel („True Peaks“) und die bekannten Nicht-Gipfel („Fake Peaks“) werden aus der angegebenen JSON-Datei geladen.
2. **Gipfelsuche:** Der Gipfelfindungsalgorithmus wird mit den in der Konfiguration definierten Schwellenwerten (z.B. für Prominenz und Dominanz) auf den Geodaten ausgeführt.
3. **Normalisierung:** Die gefundenen Gipfel werden in ein einheitliches Datenformat überführt, um den Vergleich mit den realen Gipfeln zu ermöglichen.
4. **Matching:** Die gefundenen Gipfel werden mit den realen Gipfeln abgeglichen. Ein

gefundener Gipfel gilt als TTreffer"(*True Positive*), wenn er sich innerhalb eines definierten Radius (*Matching-Area*) um einen realen Gipfel befindet. Gefundene Gipfel, die keinem realen Gipfel zugeordnet werden können, gelten als *False Positives*. Reale Gipfel, die nicht gefunden wurden, sind *False Negatives*.

5. **Auswertung und Statistik:** Aus den Ergebnissen des Matchings werden Metriken wie Präzision (*Precision*), Sensitivität (*Recall*) und der F1-Score berechnet. Zusätzlich werden die durchschnittlichen Abweichungen für Position, Höhe, Prominenz und Dominanz ermittelt.

6. **Ausgabe und Export:** Die berechneten Statistiken und Ergebnisse werden in der Konsole ausgegeben und zur späteren Analyse in einer JSON-Datei gespeichert.

maybe hier diagramm über genaue ausführreichenfolge und datenübergabelogik und dependencies?

1.4 Testen des Algorithmus

In diesem Unterkapitel werden die Test-Szenarien auf die der Tester getestet wird beschreiben. Außerdem wird die Beschaffung der Testdaten erläutert und die Testergebnisse bewertet.

1.4.1 Allgemeine Bewertung

Der Algorithmus ist bereits in der Lage Gipfel zu erkennen. Er hat jedoch Probleme mit den Randwerten und der Dominanz sowie Prominenzberechnung. Daher müssen diese Programmbausteine untersucht und verbessert werden. Besonders auffällig sind auch die Hohen Höhenabweichungen bei der Wetterstein und der Südschwarzwaldkarte. Um diese bei künftigen Tests als Fehler auszuschliesen sollten die Geodaten des Copernicus-Programm nicht mehr genutzt werden.

1.4.2 Definition der Test-Szenarien

Um die korrekte Funktion des Algorithmus zu prüfen wird auf verschiedenen Kartenausschnitten getestet. Dazu wurden drei Kartenauschnitte ausgewählt, die verscheidenen Fälle abbilden in welchen der Algorithmus eingesetzt werden kann.

bild Wetterstein

Der erste Kartenausschnitt konzentriert sich auf das Wettersteingebirge (Alpen) bei Gamisch-Partenkirchen und große Teile des Karwendelgebirges nördlich von Innsbruck (Österreich), sowie viele umliegende Berge. Dieser Kartenausschnitt wurde ausgewählt, da er viele Bergrücken und Grate beinhaltet, die in diverse Richtungen abfallen und denoch weitgehend freistehende Berge wie die Große Arnspitze oder den Vorderskopf beinhaltet. Dieser Kartenabschnitt bietet zudem vielfältige Berghöhen und gleichzeitig viele Gipfel, die Höhentechnisch nahe bei einander sind.

umformulieren

bild Feldberg Kartenausschnitt

Der zweite Kartenausschnitt zentriert sich um den höchsten Berg des Schwarzwaldes, den Feldberg. Dieser Bereich wurde gewählt, da der Algorithmus bis jetzt weniger in Mittelgebirgen getestet wurde. In Mittelgebirgen sind die Hänge meist weniger steil und die Gipfel weniger eindeutig. Die Whal fiel auf die Region um den Feldberg, da diese Region gut kartographiert ist und daher die Gipfel schon genau bestimmt sind. Außerdem gleicht der Feldberg auf seinem Gipfel mehr einer Hochebene als einem herkömlichen Gipfel. Dieses Terrain ist daher ein gutes Gegenstück zu den schroffen und unebenen nördlichen Kalkalpen.

bild Süd-schwarzwald

Der letzte Kartenausschnitt ist ein großer Kartenausschnitt des Südschwarzwalds. Die Wahl fällt auf den Südschwarzwald, da er sehr vielseitig ist. Er fällt in richtung Süden und Westen steil ins Rheintal ab und wandelt sich in den Osten in eine Hügellandschaft. Dieser Test soll, durch das einschließen des Rheintals bei Schaffhausen außerdem zeigen, wie sich der Algorithmus auf Gebiet bedienen lässt, wo nicht alle Gipfel in der Datenquelle zwingend bekannt sind.

Es wurde kein Abschnitt in den Schweizer Alpen gewählt, da der Algorithmus dort schon in getestet wurde.

zitat zu Tester-Studienarbeit

Die Vergleichsbereich werden bei den Tests jeweils jeweils einmal als $\sqrt{2}$ * *Pixelgröße*, *100m* und *minimale Dominanz* gewählt. Das sorgt dafür, dass schnell erkennbar ist welche Gipfel auf welcher Genauigkeit erkannt wurden. Dabei beschreibt $\sqrt{2}$ * *Pixelgröße* die maximale Genauigkeit mit der eine pixelbassierte Karte einen Gipfel voraussagen kann. Die *100m* sind der Bereich, den wir als gewünschte Genauigkeit des Algorithmus definiert haben. Die *minimale Dominanz* beschreibt nach **??** den maximalen Bereich in dem wir die realen Gipfel den gefundenen zuordnen können.

1.4.3 Durchlauf der Tests

Nachdem die Testdaten erstellt wurden, wurden die Tests mit verschiedenen Konfigurationen des Algorithmus durchgeführt. Dabei wurden verschiedene Werte für die Schwellenwerte der Prominenz und Dominanz getestet, um zu sehen, wie sich diese auf die Genauigkeit der Gipfelerkennung auswirken. Dabei ist aufgefallen, dass die Datenqualität aprupt abnimmt, wenn der Schwellenwert für die Prominenz und der Dominanz zu neidrig angesetzt werden. Um einerseits den Gipfelfinder in schwerem Gebiet zu Testen und gleichzeitig die Datenqualität zu erhalten, wurde bei keinem Test der Schwellenwert für die Prominenz unter 100m und der Schwellenwert für die Dominanz unter 1000m gesetzt. Es wurden aber auch Tests mit höheren Schwellenwerten durchgeführt, um zu sehen, wie sich diese auf die Genauigkeit auswirken.

1.4.4 Beschaffung der Testdaten

Die Testdaten wurden aus verschiedenen Quellen beschafft. Die Geodaten (GeoTIFF-Dateien) wurden von verschiedenen Quellen beschafft. Für die Alpen wurden die Daten des Copernicus Programms der EU bezogen. Diese Entscheidung wurde Aufgrund der Ländergrenze zwischen Deutschland und Österreich, welche sich in dem Kartenausschnitt befindet, getätigt. Als Kartenmodell wurde hier die „Digital Elevation Model (DEM) for Europe at 30 meter“ gewählt. Aus ihr können Kartenausschnitte von Europa im mit einer Auflösung von 30m pro Pixel heruntergeladen werden. Um die Performance des Tests zu verbessern wurde der Kartenausschnitt auf eine Pixelgröße von ca. 90m umgerechent. Die Kartendaten für den Südschwarzwald wurden auch aus dieser Quelle gewonnen. Das liegt daran, dass auch Teile der nördlichen Schweiz bei Schaffhausen abgebildet werden. Für den Südschwazwaldtest wurde die Kartengenauigkeit nich verringert, sondern auf 30m belassen.

abkürzung

Für den Feldberg wurden die Daten von der Landesvermessung Baden-Württemberg bezogen. Der Grund hierfür ist, dass die Daten dort genauer sind. Alle Daten wurden in einer Auflösung von 0.25m pro Pixel bezogen. Da diese Auflösung zu hoch für die Testzwecke ist, wurden die Daten auf eine Auflösung von 10m pro Pixel heruntergerechnet.

Die Daten über die realen Gipfel wurden aus verschiedenen Quellen bezogen. Da in den Daten nicht flächendeckend die Prominenz und Dominaz definiert wruden, mussten bei der Erstellung der Realdatensätze verschiedene API-Quellen und manuelle Suche kombiniert werden. Zum einen wurden die Daten von der OpenStreetMap API bezogen. Diese Daten wurden mit Hilfe der Overpass API abgefragt. Es wurden alle Punkte mit dem Tag *natural=peak* innerhalb der Kartenausschnitte abgefragt. Diese Daten wurden wurden zunächst mit den Daten von Wikidata abgeglichen und durch diese ergänzt. Wikidata bietet eine API an, über die für bestimmte Koordinaten die Gipfel abgefragt werden können. Mit diesem Abgleich konnten über mehrere Kartenbereichen in den Alpen im Durchschnitt 80% der Gipfel mit Prominenz über 100m gefunden werden. Dies leigt nicht daran, dass die Gipfel selbst unbekannt sind, sondern daran, dass die Prominenz und Dominanz nicht in den Datenquellen definiert wurden. Daher wurden die Daten von Hand ergänzt und auf Vollständigkeit überprüft. Zum besseren Analyseren der Tests wurden die Gipfel, die nicht den Kriterien für einen Gipfel entsprechen, als *Fake Peaks* definiert und in die Testdaten aufgenommen. Es wurden Punkte ausgewählt, die in der Nähe von realen Gipfeln liegen, aber nicht die Kriterien für einen Gipfel erfüllen oder von den APIs als Gipfel, die den Bedingungen nicht entprechen erkannt wurden. Diese Daten helfen bei dem schnellen finden von Fehlern des Algorithmus, da sie zeigen, welchen Gipfel der Algorithmus fälschlicherweise als Gipfel identifiziert hat und nicht nur seine Koordinaten. Um die Daten manuell zu ergänzen und zu überprüfen muss die Karte von Hand durchsucht werden. Dabei stellt sich die Frage, welche Karte verwendet werden sollte, um die genausten testdaten zu erhalten. Dabei spielen Faktoren wie die allgemeine Genauigkeit der Höhenlinien und Anzahl der makierten Gipfel eine Rolle. Die Höhenlinien sind wichtig um die Prominenz abzuschätzen, wenn keine Realwerte dafür existieren. Bei wie vielen Gipfel keine Realwerte in den Datenquellen existieren hängt von vielen Faktoren ab. Dabei gilt meist, dass desto berühmter, höher die Gipfel und desto höher deren Prominenz und Dominanz ist umso wahrscheinlicher existieren Realwerte für diese Berge und umso besser sind diese. Ersteres spielt nach der Erfahrung eine größere Rolle. So ist beispielsweise die Höhe der Zugspitze in der Wikidata Datenbank auf Centimeter genau definiert, während die Höhe der meisten anderen Gipfel nur auf Meter genau definiert ist. Es gibt aber auch Fälle, wie beispielsweise den Daniel, bei der die Höhe nur auf 10 Meter genau definiert ist, obwohl

sie ein eine große Prominenz von über 500m besitzt.

Es wurden verschiedene Karten verglichen, wie die OpenStreetMap Karte, die Karte von Google Maps und die Karte von Bing Maps. Es wurde festgestellt, dass die OpenStreetMap Karte die genausten Höhenlinien und die meisten markierten Gipfel enthält. Daher wurde sie als Hauptquelle für die manuelle Ergänzung der Testdaten verwendet. Die Werte für die Prominenz und Dominanz wurden dabei durch Abschätzen der Höhenlinien auf der Karte ergänzt, wenn nicht schon durch die APIs gegeben.

1.4.5 Bewertung der Testergebnisse

Nach dem durchlaufen der Tests wurden die Ergebnisse ananlysiert und bewertet. Dabei wurde nicht nur die Genauigkeit der Gipfelerkennung bewertet, sondern auch die durchschnittlichen und extremen Abweichungen für Position, Höhe, Prominenz und Dominanz, sowie die Laufzeit. Dabei werden zunächst alle Test-Szenarien einzeln bewertet und anschließend die Ergebnisse zusammengefasst. Dabei wurden zur besseren Vergleichbarkeit alle Tests mit einer Boder width von 50px ausgeführt. Außerdem wurde bei allen Test mit einer minimalen orographischen Dominanz von 0% gerechnet. Letzteres hat den Grund, dass die orographische Dominanz aus der Prominenz und der Höhe berechnet werden kann. Daher ist es einfacher zum testen dieser Funktion keine Feldtests durchgeführen, sondern einfache Unittests zu schreiben.

border with auf deutsch formulieren

verweis auf unittests

Wetterstein-Tests

Im Wettersteingebirge und Umland wurden mit dem Algorithmus alle Berge mit einer minimalen Höhe von 0m, Prominenz von über 500m und Dominanz von minimal 2km gesucht. Diese Werte wurden gewählt, damit die Zahl der zu untersuchenden Gipfel nicht zu groß ausfällt. In diesem Test wurden 95% der realen Gipfel auf 1000m korrekt erkannt (Precision) und 68% der gefundenen Gipfel waren in 1000m um einen realen Gipfel, der den Kriterien entsprach (Sensitivity). Bei einem Vergleichsbereich von 100m werden noch 60% der Gipfel richtig erkannt. Auf den Pixel genau (ca. 65m) werden nur 55% der Gipfel erkannt. Die Restlichen Metriken sind hier in einer Tabelle zu finden:

abgleich wirklich geleich emtriken wie in 4.2

diese überleitung umformulieren

In diesen Allgemeinen Testergenissen ist gut zu erkennen, dass der Tester weniger Pro-

matching areas auf pixelwidth/2 bei pixelgenauc und rest auf 100m und 1000m circular

Tabelle 1.1: Metriken Wetterstein 100m Abstand

Metrik	Wert
Precision	0.60
Sensitivity	0.43
F1-Score	0.50
Durchschnittlicher Positionsfehler (m)	30.83
Durchschnittlicher x-Fehler (m)	23.35
Durchschnittlicher y-Fehler (m)	17.44
Durchschnittlicher Höhenfehler (m)	26.83
Durchschnittlicher Dominanzfehler (m)	641.77
Durchschnittlicher Prominenzfehler (m)	97.33

Tabelle 1.2: Metriken Wetterstein 45m Manchattendistanz

Metrik	Wert
Precision	0.55
Sensitivity	0.39
F1-Score	0.46
Durchschnittlicher Positionsfehler (m)	26.06
Durchschnittlicher x-Fehler (m)	21.39
Durchschnittlicher y-Fehler (m)	12.65
Durchschnittlicher Höhenfehler (m)	26
Durchschnittlicher Dominanzfehler (m)	677.45
Durchschnittlicher Prominenzfehler (m)	91.82

Tabelle 1.3: Metriken Wetterstein 1000m Abstand

Metrik	Wert
Präzision	0.95
Sensitivität	0.68
F1-Score	0.79
Durchschnittlicher Positionsfehler (m)	149.88
Durchschnittlicher x-Fehler (m)	130.65
Durchschnittlicher y-Fehler (m)	51.19
Durchschnittlicher Höhenfehler (m)	32.47
Durchschnittlicher Dominanzfehler (m)	1040.35
Durchschnittlicher Prominenzfehler (m)	98.58

bleme damit hat Gipfel zu erkennen, sondern deren Prominenz und Dominanz richtig zu bestimmen. Das zeigt sich in den für die gewählten Schwellenwerte hohen durchscnittlichen Prominenz und Dominanz. Um nun genauer zu erkennen, was für die Abweichungen verantwortlich ist, müssen die einzelnen Werte untersucht werden. Dabei fällt auf, dass die meisten Berge eine unerwartet hohe Dominanzabweichung haben. Dabei gibt es auch ein paar Gipfel, die besonders hohe oder niedrige Abweichungen haben. Diese sind für die Bestimmung der Fehlerursache wichtig. Der Gipfel mit der höchsten Abweichung ist die Birkkarspitze (11,5km) und der Gipfel mit der niedrigsten ist der Hochplattig (36m). Eine höhere Abweichung der Dominanz vom Realwert lässt sich bei der Birkkarspitze zwar alleine durch die Hohe Dominanz von 33km erklären, aber 11,5km ist dennoch zu hoch um nicht auf einen Fehler im Höhenmodell oder Algorithmus hinzudeuten. Um das Problem besser einordnen zu können, werden die Gipfel und ihre Possition, sowie nächst höheren Punkte miteinander verglichen. Das folgende Diagramm 1.1 zeigt den Kartenbereich und vier Gipfel die so aufbereitet wurden.

Zusätzlich sind die relativen Fehler der Dominanzen in dem Diagramm 1.4 zu sehen. Hieran ist gut zu erkennen, dass der Hochplattig der einzige Berg ist, desen Dominanz nur in Y-Richtung bestimmt wurde.

schlussfoldern, das berechnung der Distanz Müll

Ein anderes Problem ist, dass der Algorithmus Gipfel erkennt, die in Realität nicht existieren. Das passiert nur in der Nähe des Kartenrandes. Das liegt daran, dass kleine Er-

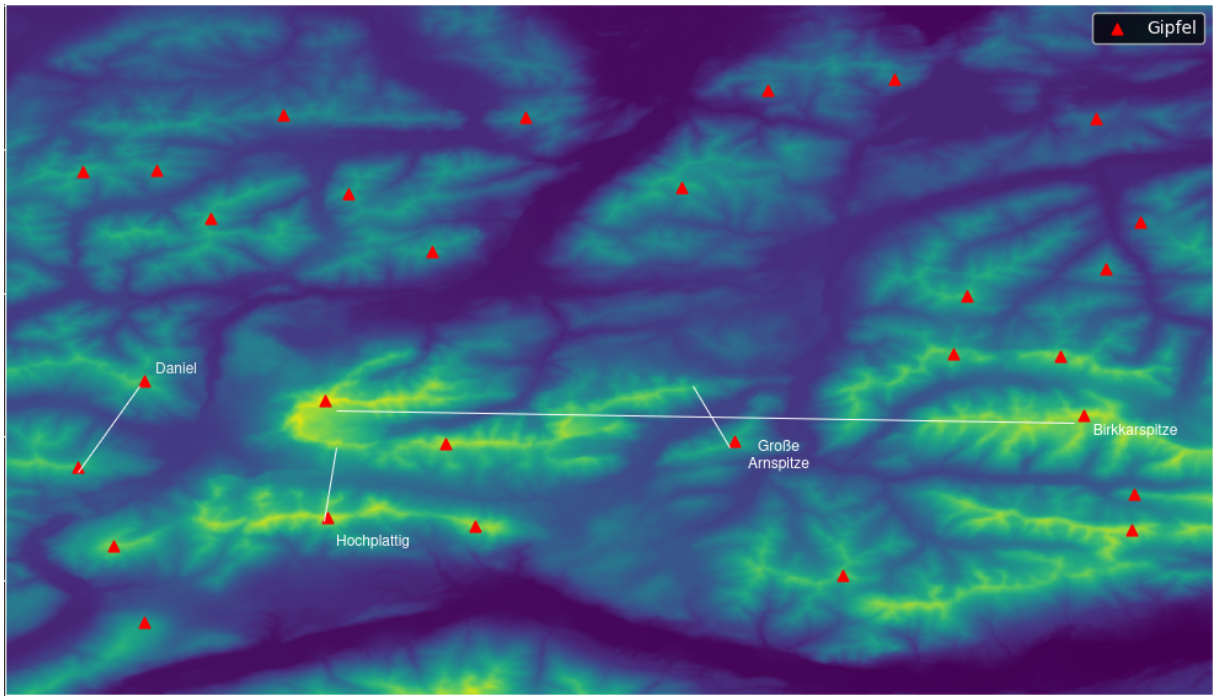

Abbildung 1.1: Dominanz von Daniel, Birkkarspitze, Hochplattig und Große Arnspitze

Tabelle 1.4: Relative Fehler pro Gipfel

Gipfel	Wert (%)
Birkkarspitze	34,84%
Hochplattig	0,6%
Daniel	36,0%
Große Arnspitze	16,9%

hebungen am Hang eines Berges nahe des Kartenrandes nicht als Gipfel ausgeschlossen werden können. In diesem Fall wird ihnen der Hang des gesammten Berges bei der Prominenzberechnung angerechent. Um dies zu verhindern wurde bereits die *border width* eingeführt. Diese scheint aber nicht richtig zu funktionieren, die Randwertfehler nun am Rand der *border width* auftretten.

diagram mit Zylinder oder berg und dann ne border durch/ höhenlinien an border

das border width problem diagramm

Der Algorithmus erkennt einzelne Gipfel nicht als Gipfel. In diesem Test-Szenario ist das der kleine Solstein bei Innsbruck. Der Grund hierfür ist, dass seine Prominenz als unter 500m vorhergesagt wird, obwohl sie bei 517m liegt. Dieses Problem lässt sich auf die Kartenqualität zurückführen.

Wenn ein Gipfel in dem Vergleichsbereich um einen realen Gipfel liegt, heißt das nicht, dass der Algorithmus den Gipfel erkennt. Das wird bei der Birkkarspitze erkenntlich, wo der

Algorithmus die mittlere Ödkarspitze als Gipfel erkannt hat, die eigentliche Birkkarspitze nicht. Dieser Fehler kann mit der genauigkeit der Karte erklärt werden.

Feldberg-Test

Beim Feldberg-Test wurden mit dem Algorithmus alle Berge mit einer minimalen Höhe von 0m, Prominenz von über 100m und Dominanz von minimal 200m gesucht. Diese Werte wurden gewählt, damit die Zahl der zu untersuchenden Gipfel auf diesem kleineren Kartenasuschnitt nicht zu gering wird. Mit diesen Parametern wurden 91% der realen Gipfel auf 100m korrekt erkannt und 67% der gefunden Gipfel waren reale Gipfel. Dies zeigt ein ähnliches Bild, wie auch schon der Wetterstein-Test. Der Algorithmus findet zwar die meisten Gipfel, findet jedoch auch zusätzliche Gipfel nahe des Randes, die keinen realen Gipfel entsprechen. Anders als beim Wetterstein-Test bestimmt der Algorithmus beim Feldbergtest nur 18% der Gipfel auf den Pixel genau. Dies kann durch die kleinere Rastergröße von 10m und die Genauigkeit der Realwerte erklärt werden. Weitere Metriken der Tests sind in den Tabellen 1.5 und 1.6 enthalten.

Tabelle 1.5: Metriken Feldberg 5m Manchattendistanz

Metrik	Wert
Präzision	0.18
Sensitivität	0.13
F1-Score	0.15
Durchschnittlicher Positionsfehler (m)	7.11
Durchschnittlicher x-Fehler (m)	4.84
Durchschnittlicher y-Fehler (m)	5.14
Durchschnittlicher Höhenfehler (m)	5.0
Durchschnittlicher Dominanzfehler (m)	30.8
Durchschnittlicher Prominenzfehler (m)	1.5

An diesem Testfall ist gut zu erkennen, dass die Karten des Vermessungsamtes Baden Württemberg genauer sind als die öffentlich zugänglichen Daten des Copernikusprogramms. Dies spiegelt sich in den geringeren Höhen- sowie Positionsfehlern wieder. Außerdem zeigen diese Tests, dass der Algorithmus die Gipfel nicht in eine bestimmte

Tabelle 1.6: Metriken Feldberg 100m Abstand

Metrik	Wert
Präzision	0.91
Sensitivität	0.67
F1-Score	0.77
Durchschnittlicher Positionsfehler (m)	30.86
Durchschnittlicher x-Fehler (m)	20.15
Durchschnittlicher y-Fehler (m)	19.91
Durchschnittlicher Höhenfehler (m)	2.1
Durchschnittlicher Dominanzfehler (m)	21.15
Durchschnittlicher Prominenzfehler (m)	3.3

Richtung genauer oder ungenauer bestimmt, anders als es die Testdaten des Wetterstein-gebirge vermuten lassen.

geringe Dominanz sowie Prominenzabweichungen

Südschwarzwald-Test

Beim Südschwarzwald-Test wurden mit dem Algorithmus alle Berge mit einer minimalen Höhe von 0m, Prominenz von über 100m und Dominanz von minimal 2000m gesucht. Dabei wurden wie bei den letzten Tests auf 1000m Genauigkeit über 90% der Gipfel richtig bestimmt und 79% der gefundenen Gipfel waren real. Die genauen Testergebnisse können in den Tabellen 1.9, 1.8 und 1.7 abgelesen werden. Dabei fällt auf, dass die Höhenungenauigkeiten ungefähr einem Drittel des Wetterstein-Tests entsprechen. Das bedeutet, dass es wahrscheinlcih ist, dass das verringern der Auflösung einen Einfluss auf die Ergebnisse hatte. Zudem kann erkannt werden, dass mit schlechterer Auflösung die Dominanzen ungenauer werden. Es kann beobachtet werden, dass der Prominenzfehler ungefähr dem doppelten Höhenfehler entspricht.

Laufzeittests in 5 zu finden

1.5 Bewertung des Testers

Der Tester ist gut geeignet um diesen Algorithmus zu Testen. Er implementiert eine Logik zur Überprüfung der Genauigkeit und erkennt, welche Gipfel gefunden werden. Die

Tabelle 1.7: Metriken Südschwarzwald 15m Manchattendistanz

Metrik	Wert
Präzision	0.28
Sensitivität	0.23
F1-Score	0.25
Durchschnittlicher Positionsfehler (m)	5.26
Durchschnittlicher x-Fehler (m)	3.95
Durchschnittlicher y-Fehler (m)	2.27
Durchschnittlicher Höhenfehler (m)	7.7
Durchschnittlicher Dominanzfehler (m)	1333.92
Durchschnittlicher Prominenzfehler (m)	14.2

Tabelle 1.8: Metriken Südschwarzwald 100m Abstand

Metrik	Wert
Präzision	0.72
Sensitivität	0.6
F1-Score	0.66
Durchschnittlicher Positionsfehler (m)	35
Durchschnittlicher x-Fehler (m)	22.12
Durchschnittlicher y-Fehler (m)	10.23
Durchschnittlicher Höhenfehler (m)	10.23
Durchschnittlicher Dominanzfehler (m)	997.33
Durchschnittlicher Prominenzfehler (m)	45.35

Tabelle 1.9: Metriken Südschwarzwald 1000m Abstand

Metrik	Wert
Präzision	0.94
Sensitivität	0.79
F1-Score	0.86
Durchschnittlicher Positionsfehler (m)	82.19
Durchschnittlicher x-Fehler (m)	45.58
Durchschnittlicher y-Fehler (m)	61.71
Durchschnittlicher Höhenfehler (m)	9.79
Durchschnittlicher Dominanzfehler (m)	893.68
Durchschnittlicher Prominenzfehler (m)	40.56

damit zusammenhängende Umformulierung des Gipfelsuchproblem zu einem Klassifizierungsproblem hat geholfen um schnell zu erkennen zu können, wie gut der Algorithmus Gipfel erkennt. Diese Methodik hat das manuele Vergelichen der Algorithmusergebnisse überflüssig gemacht. Die in der falschen Gipfel in der Realdaten-Datei haben bei der Diagnose von Fehlern geholfen. Dadurch konnten nicht gefundene Gipfel schneller auf einer Karte entdeckt werden.

besserer Name für die datei und konsistenz dieses namens

Die Wahl der Testdaten war nicht ideal. Es hätten weniger Geodaten des Copernikusprogramm verwendet werden sollen. Diese sind ungenauer als die des baden-württembergischen oder bayerischen Vermessungsamtes. Um diese Daten nutzen zu können hätten andere Kartenausschnitte gewählt werden müssen. Es mussten auch viele Realdaten manuell überprüft und ergänzt werden. Das hätte auch mit der Wahl einer höheren minimalen Prominenz und Dominanz verbessert werden könne, da dadurch nach weniger unbekannten Gipfeln auf den Kartenausschnitten gesucht werden würde. Das wäre im Schwarzwald nicht zielführend gewesen und somit hätten die untersuchetn Kartenausschnitte anders gewählt werden müssen.

2 Optimierung des Algorithmus

intro zu 5

2.1 Motivation

Die Analyse in Kapitel 1.4 zeigte Probleme des Algorithmus auf, die sich vor allem in Abweichungen bei der Berechnung von Dominanz und Prominenz äußern. Da diese Werte die Grundlage für die Einordnung von Gipfeln bilden, müssen sie so genau wie möglich sein.

Eine zentrale Anforderung an die Software ist ihre Zuverlässigkeit: Der Nutzer muss sich darauf verlassen können, dass der Algorithmus sämtliche relevanten Gipfel findet. Bei der Optimierung gilt daher: Es ist wichtiger, alle echten Gipfel zu finden (Vermeidung von False Negatives), als fälschlicherweise erkannte Gipfel komplett auszuschließen (Vermeidung von False Positives). Ein übersehener Gipfel wiegt schwerer als ein falscher.

Nach diesen Grundsätzen werden im Folgenden Verbesserungen vorgenommen. Ein weiteres wesentliches Ziel ist die Reduktion der Laufzeit. Nur wenn der Algorithmus effizient arbeitet, kann er in der Praxis zielführend eingesetzt werden. Dabei dürfen Optimierungen der Laufzeit die Genauigkeit der Ergebnisse jedoch nicht verschlechtern.

achtung kann wie ki klingen

2.2 Vorgehen

Um den Algorithmus besser verstehen und verbessern zu können, muss er modularisiert und übersichtlicher gestaltet werden. Dies soll mit der Hilfe eines Refactorings zu einem objektorientierten Progamm erreicht werden. Außerdem sollen die Methoden statt Tupel geordnete Daten zurückgeben.

Nach dem Refactoring werden die Prominenzberechnug und die Dominanzberechung untersucht und verbessert. Dabei wird jede Verbesserung mit Hilfe des Testers aus 1 validiert. Der Tester wird so als Tool zur Qualitätserhaltung sowie der verifikation der Verbesserungen eingesetzt.

2.3 Verbesserung der Abbruchbedingung und -logik

Die Analyse der Testergebnisse hat gezeigt, dass die Prominenzwerte teilweise fehlerhaft sind. Die Ursache liegt in einer nicht optimal implementierten Abbruchbedingung bei der Prominenzberechnung.

Die bisherige Implementierung ermittelt den Sattelpunkt zum nächstgelegenen höheren Gipfel. Dies entspricht jedoch nicht der korrekten Definition der Prominenz. Die Prominenz (auch Schartentiefe genannt) ist der Höhenunterschied zwischen einem Gipfel und dem höchstgelegenen Sattel, der ihn mit einem höheren Gipfel verbindet . Dieser Sattel gehört nicht zwangsläufig zum nächstgelegenen höheren Gipfel.

ref def Prominenz

Das Beispiel des Zugspitzecks verdeutlicht dieses Problem (siehe Abbildung 2.1). Das Zugspitzeck liegt näher am Schneefernerkopf als an der Zugspitze. Der Sattel zum Schneefernerkopf liegt auf 2699 m, während der Sattel zur Zugspitze auf 2788 m liegt. Der Sattel zur Zugspitze ist also der Sattelpunkt. Die alte Implementierung würde fälschlicherweise den Sattel zum näheren Schneefernerkopf wählen und eine Prominenz von 116 m berechnen, obwohl der korrekte Wert, basierend auf dem Sattel zur Zugspitze, nur 27 m beträgt.

Um diesen Fehler zu beheben, wird die Abbruchlogik angepasst. Die neue Abbruchbedingung beendet die Suche, sobald ein Punkt erreicht wird, der höher ist als der Ausgangsgipfel.

Dieser Ansatz hat zwei Vorteile. Erstens garantiert er das Finden des korrekten, höchsten Sattelpunktes. Da der Suchalgorithmus von einem Pixel immer zum höchstgelegenen Nachbarpixel fortschreitet (ähnlich einer Gradientenabstiegsmethode auf einem Inversrelief), folgt er dem Pfad des geringsten Höhenverlusts. Dieser Pfad führt zwangsläufig über den höchsten Sattel, bevor ein höherer Punkt erreicht wird. Zweitens wird die Berechnungseffizienz gesteigert, da die Suche abgebrochen werden kann, ohne den zugehörigen höheren Gipfel explizit finden zu müssen. Dies reduziert die Anzahl der zu untersuchenden Pixel erheblich.

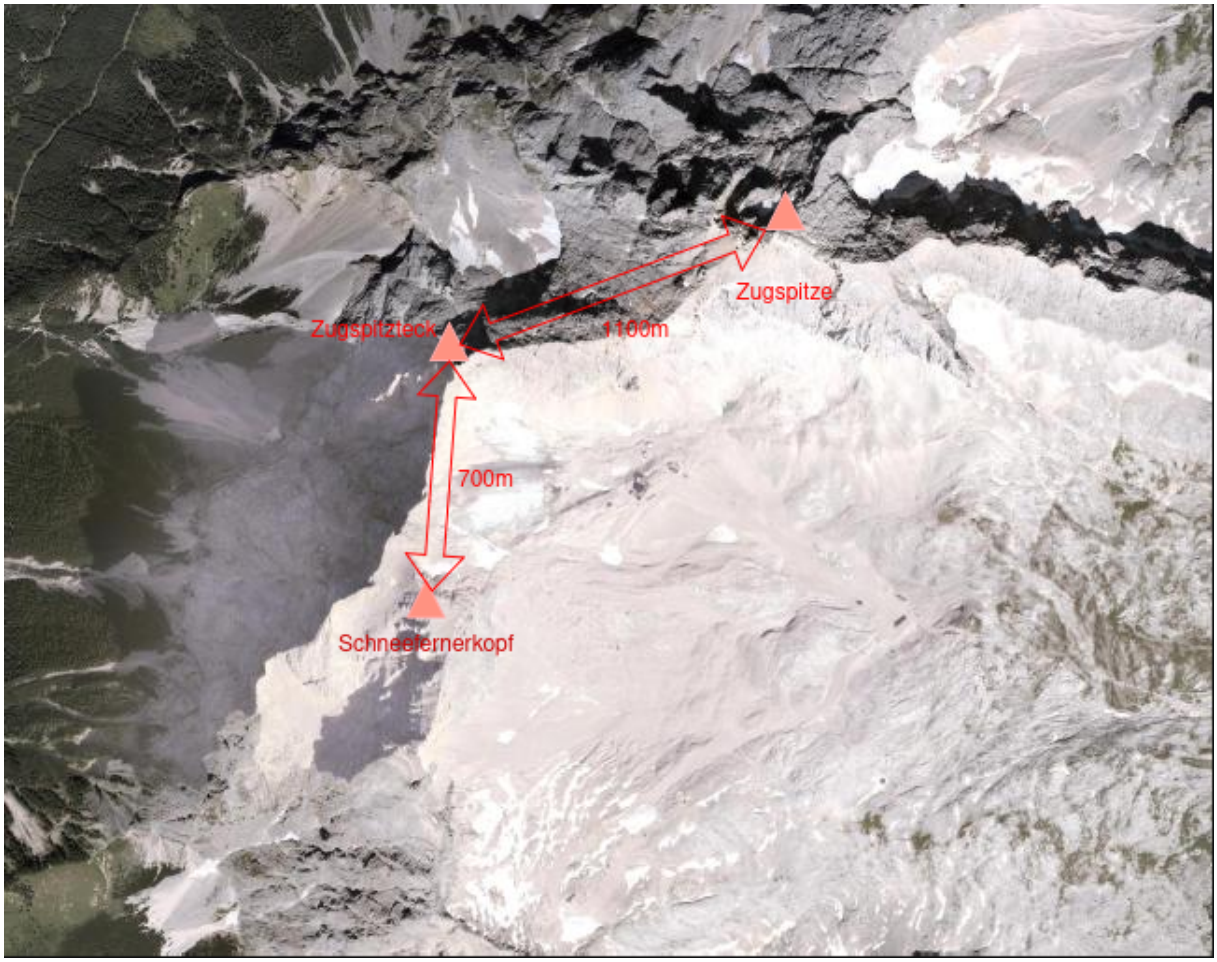

Abbildung 2.1: Satelitenbild des Zugspitzecks

2.4 Verbesserung der Distanzberechnung

Die Berechnung der Dominanz wies einen systematischen Fehler auf, der durch eine ungenaue Distanzmessung verursacht wurde. Im ursprünglichen Algorithmus wurde die Distanz zwischen dem Gipfel und dem nächsthöheren Punkt mittels der euklidischen Distanztransformation auf die Pixelkoordinaten berechnet. Dieser Ansatz, der dem Satz des Pythagoras auf einem Pixelgitter entspricht, ist jedoch aus mehreren Gründen problematisch und führt zu ungenauen Ergebnissen.

Ein grundlegendes Problem ist die inhärente Vereinfachung, die Geodaten als eine flache, zweidimensionale Ebene zu behandeln. Die euklidische Distanz ignoriert die Krümmung der Erdoberfläche vollständig. Während dieser Fehler bei kleinen Kartenausschnitten vernachlässigbar sein mag, wächst er mit zunehmender Entfernung signifikant an und führt zu einer systematischen Unterschätzung der wahren geodätischen Distanz.

Des Weiteren basiert die pixelbasierte Berechnung auf der fehlerhaften Annahme, dass jedes Pixel einer quadratischen Fläche mit konstanter Kantenlänge in der Realität entspricht. Dies trifft auf viele Geodatenprojektionen nicht zu. Insbesondere bei Daten im weit verbreiteten WGS84-Format, das auf geografischen Koordinaten basiert, ist diese Annahme falsch. Der durch einen Längengrad abgedeckte Abstand in Metern schrumpft von über 111 km am Äquator auf null an den Polen. Die Anwendung einer euklidischen Metrik auf solche Koordinaten ist daher mathematisch inkorrekt und führt zu stark verzerrten Distanzwerten, die je nach geografischer Breite variieren.

Zusammengenommen führen diese Faktoren zu einer systematischen Verfälschung der Dominanzwerte, deren Größe und Richtung von der Position auf der Erde, der Projektion des Datensatzes und der Ausdehnung des Kartenausschnitts abhängen. Um diese Fehlerquellen zu eliminieren und eine präzisere sowie universell anwendbare Berechnung zu gewährleisten, wurde die Methode zur Distanzmessung grundlegend überarbeitet. Anstelle der pixelbasierten euklidischen Distanz wird nun die Haversine-Formel verwendet. Diese Formel berechnet den Großkreisabstand zwischen zwei Punkten auf einer Kugel (der Erde) anhand ihrer geografischen Koordinaten.

$$d = R \cdot \arccos(\sin(\phi_1)\sin(\phi_2) + \cos(\phi_1)\cos(\phi_2)\cos(\Delta\lambda)) \tag{2.1}$$

Hierbei sind ϕ_1, ϕ_2 die Breitengrade, $\Delta\lambda$ die Differenz der Längengrade und R der Erdradius.

Die Implementierung der Dominanzberechnung wurde entsprechend angepasst. Der neue Algorithmus identifiziert zunächst alle Punkte im Datensatz, die höher sind als der betrachtete Gipfel. Anschließend wird für jeden dieser höheren Punkte die geodätische Distanz zum Gipfel mittels der Haversine-Formel berechnet. Die Dominanz ist dann die kleinste dieser berechneten Distanzen. Dieses Vorgehen stellt sicher, dass die Dominanz als tatsächliche Entfernung auf der Erdoberfläche und nicht als Distanz im Pixelraum gemessen wird, was die Genauigkeit der Ergebnisse erheblich verbessert.

2.5 Neubewertung des Randwertproblems

Ein zentrales Problem bei der Analyse von topografischen Daten in begrenzten Kartenausschnitten ist das Randwertproblem. Es entsteht, weil die topografischen Informationen an den Grenzen des Datensatzes enden und somit eine vollständige Analyse der Umgebung für Punkte in Randnähe unmöglich ist. Ein Hang, der in den Kartenausschnitt hineinragt, könnte fälschlicherweise als eigenständiger Berg klassifiziert werden, da der Algorithmus nicht erkennen kann, dass es sich lediglich um den Ausläufer eines außerhalb liegenden, höheren Berges handelt. Der höchste Punkt dieses Hangs innerhalb des Ausschnitts würde fälschlicherweise als Gipfel identifiziert.

Um dieses Problem systematisch zu adressieren, wird die Konzepte der *minimalen Prominenz* und *minimalen Dominanz* eingeführt. Diese Metriken quantifizieren die untere Schranke für die wahren Prominenz- und Dominanzwerte, die allein aus den Daten des Kartenausschnitts herleitbar ist. Sie repräsentieren eine konservative Schätzung unter der Annahme, dass sich direkt außerhalb der Kartengrenze eine unendlich hohe Bergkette befindet.

Die Abbildung 2.2 zeigt die reguläre Berechnung von Prominenz und Dominanz in einem idealisierten, unbegrenzten Gelände. Die Dominanz ist der Abstand zum nächstgelegenen höheren Punkt, und die Prominenz ist die Höhendifferenz zum maßgeblichen Sattel.

Wird das Gelände jedoch durch einen Kartenausschnitt begrenzt, wie in Abbildung 2.3 dargestellt, ändern sich die garantierbaren Werte. Die minimale Dominanz wird als das Minimum aus zwei Werten definiert: der innerhalb des Ausschnitts berechneten Dominanz und der direkten Distanz des Gipfels zum Kartenrand. Diese Metrik wurde eingeführt, da ein höherer Punkt unmittelbar außerhalb des sichtbaren Kartenbereichs existieren könnte, was die wahre Dominanz limitieren würde. Analog dazu ergibt sich die minimale Prominenz als das Minimum aus der regulär berechneten Prominenz und der Höhendifferenz des Gipfels zum höchsten Sattel, der zum Kartenrand führt. Dieser RRandsattelrrepräsentiert den höchsten Punkt, der mindestens überwunden werden muss, um den Kartenausschnitt zu verlassen, und begrenzt somit die nachweisbare Prominenz. Diese Werte sollen zeigen in wie aussagekräftig der aus dem kartenausschnitt errechnete Dominanz- und Prominen-

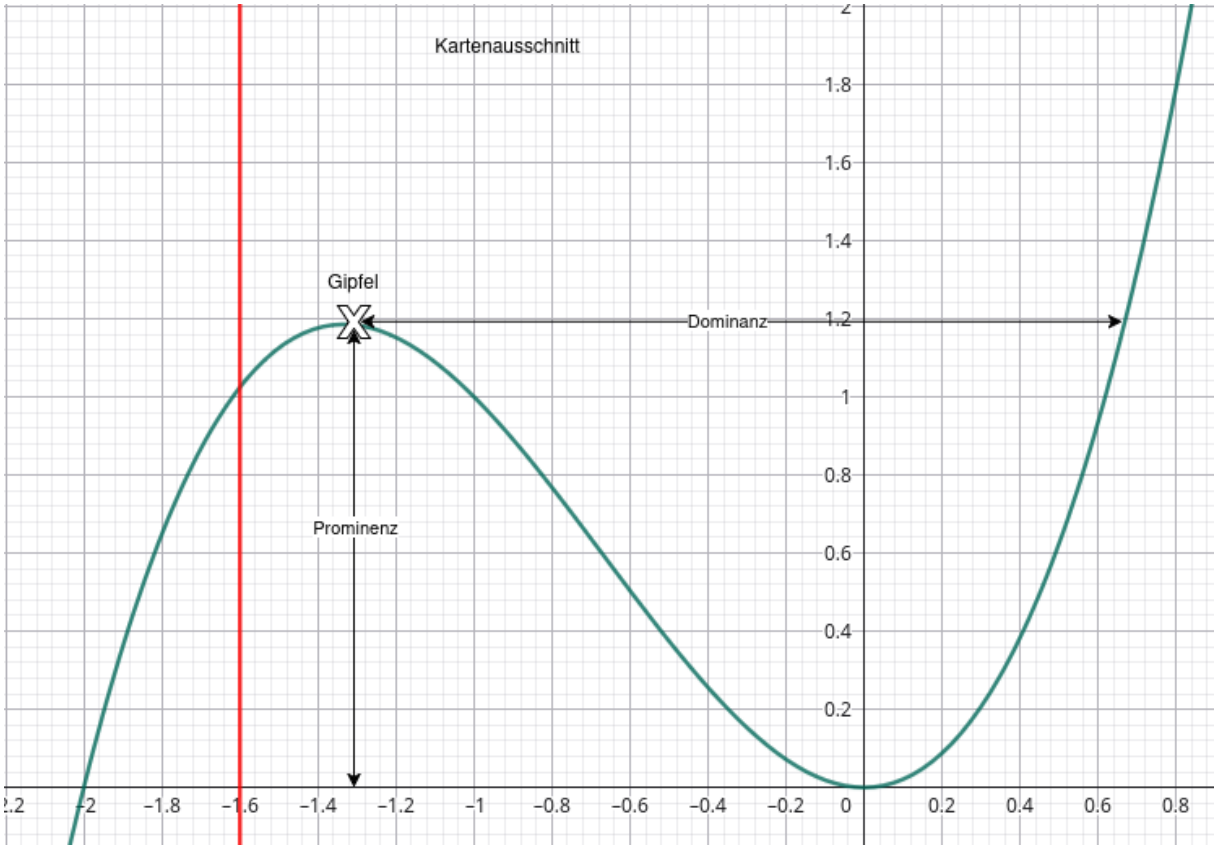

Abbildung 2.2: 2D-Geländeprofil zur Illustration von Prominenz und Dominanz eines Gipfels

falls in ui verwendet: Sicherheitswert in Prozent der Berechnugn der Dominaz und Prominenz

zwert ist.

Theoretsich lässt sich so auch der pauschale Ausschluss von Werten innerhalb eines Randabstands anders beschreiben. Der Randabstand von x Pixeln lässt sich in die Bedingung, dass die minimale Dominanz kleiner ist als das Produkt aus x und der Bodenauflösung (in Meter pro Pixel) ist umformulieren. Dies wird jedoch im Algorithmus umgesetzt, weil dadurch Randfälle wie zum Beispiel ein größerer Randabstand als Dominanzgrenzwert berücksichtig werden müssten. Im Algorithmus wird weiterhin mit einem pixelbassierten Abstand gerechnet.

Es wurde untersucht in wie weit es die Laufzeit senkt anstatt bei der Bestimmung der lokalen Hochpunkte anstatt der Setzung aller Randwertpunkte auf 0 eine Possitionsbassierte Schranke zu verwenden, die alle Gipfel aussortiert,d ie sich im Randbereich befinden. Nach mehreren Tests ist hier jedoch keine Verbesserung der Laufzeit erkennbar gewesen. Daher wurde der alte übersichtlichere und erprobte Ansatz weiter verwendet.

Eine alternative Implementierung des Randwertbereichs wäre auf Basis der minimalen

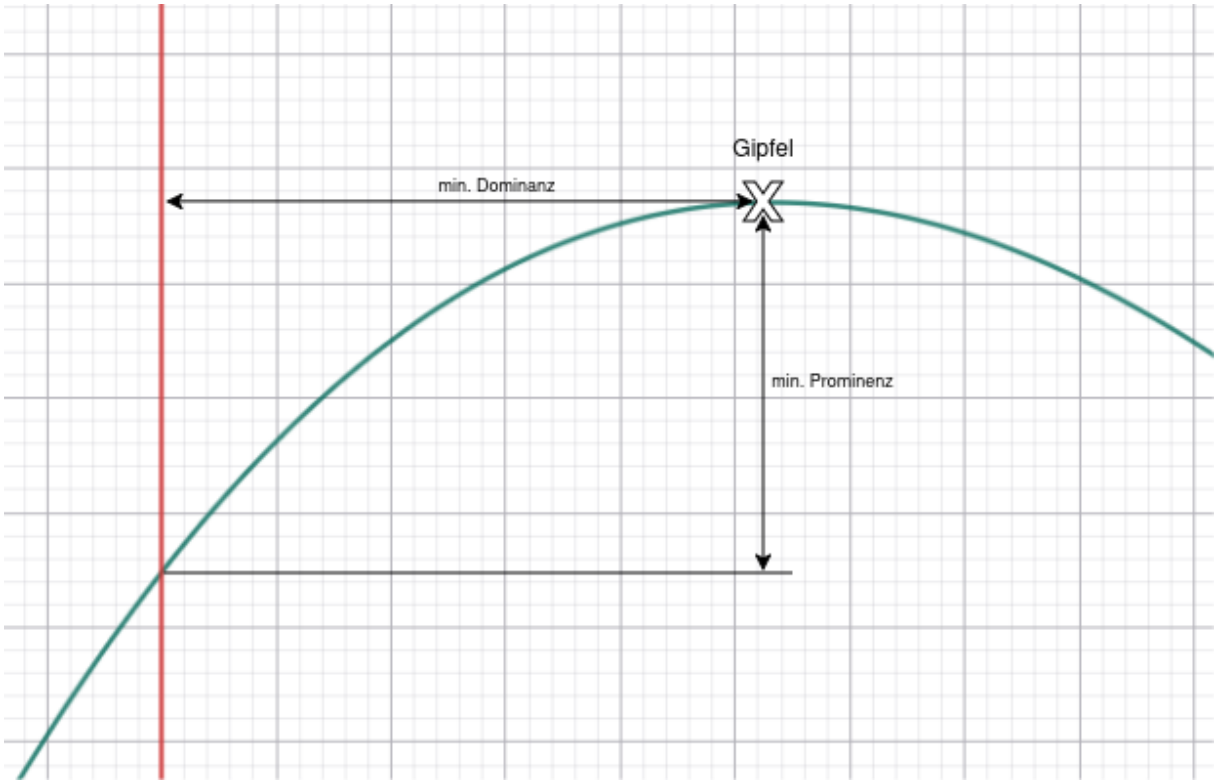

Abbildung 2.3: 2D-Geländeprofil zur Illustration der minimalen Prominenz und Dominanz bei begrenztem Kartenausschnitt.

Prominenz und Dominanz. Dabei würde alle Gipfel aussortiert werden, deren minimale Prominenz oder Dominanz unter dem jeweiligen Schwelenwert liegt. Diese Methode der Randwertbehandlung ist für diesen Algorithmus nicht zielführend, da dadurch mehr falschnegative Werte entstehen, die wir eigentlich verhindern wollen. Darum wurde sie auch nicht implementiert.

Um das Problem der Gipfel am Kartenrand zu lösen musste an einer anderen Stelle angesetzt werden. Es hat sich herausgestellt, dass wenn der Randbereich auf Null gesetzt wird und nun auf genau diesen Geodaten eine Prominenzberechung durchgeführt wird die Daten alle Werte im Randbereich ignorieren, was nicht der Sinn des Randwertes ist (wie in 2.4). Der Randwert sollte eigentlich einen Puffer geben, damit die Prominenz und die Dominanz besser berechnet werden können und nicht den Kartenbereich ohne Vorteil verkleinern.

Wenn nun jedoch die ursprünglichen Geodaten zur Prominenz und Dominanzbestimmung verwendet werden, werden die Gipfel auf dem Kartenrand nicht mehr falsch entdeckt

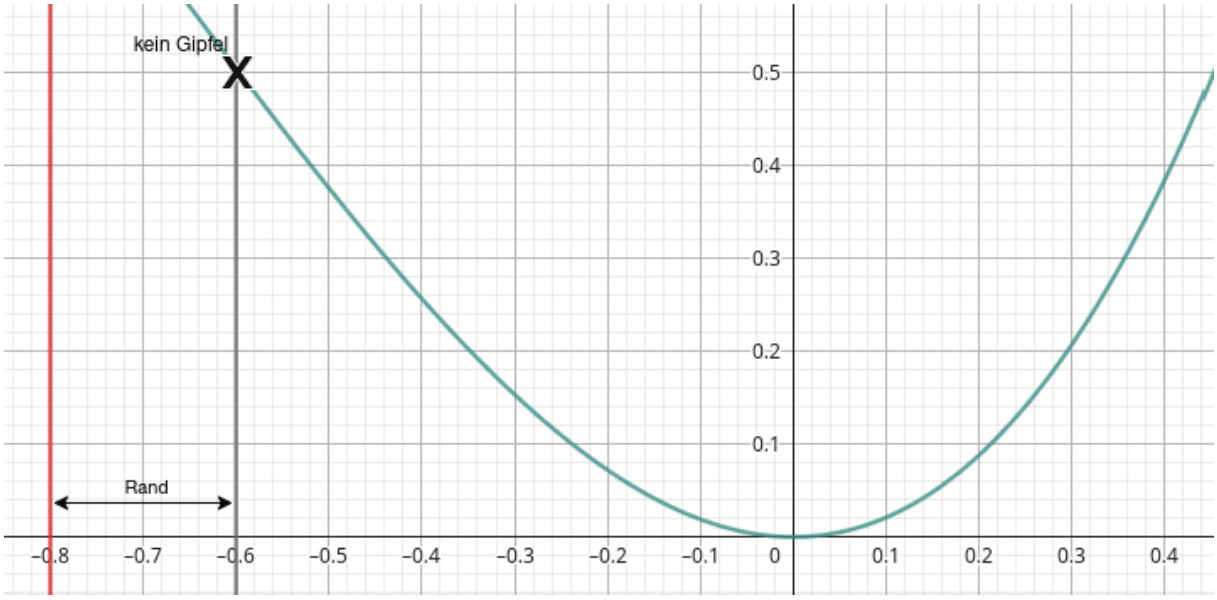

Abbildung 2.4: 2D-Geländeprofil zur Illustration der minimalen Prominenz und Dominanz bei begrenztem Kartenausschnitt.

(2.5).

2.6 Verbesserung der Maximabestimmung

performance optimierungen in eigenenm section (die size wurde erhöht bei maximumfilter)

2.7 Einordnung der Laufzeitverbesserungen

Um die Laufzeitverbesserungen zu bewerten, wurden die Laufzeiten der verschiedenen Testfälle vor und nach den Optimierungen gemessen. Es wurden die gleichen Testfälle wie in 1.4 verwendet, um eine konsistente Vergleichsbasis zu gewährleisten. Um die vergelichbarkeit der Laufzeiten zu gewährleisten, wurden die Tests auf der gleichen Hardware durchgeführt und die gleichen Softwareumgebungen verwendet. Die Test wurde auf einem Computer mit einen Ryzen 7 5700X Prozessor und 32GB RAM durchgeführt. Als Betriebssystem wurde Nobara Linux 43 verwendet. Die Laufzeiten wurden mit der Python-Bibliothek *time* gemessen, indem die Start- und Endzeitpunkte der Algorithmusausführung erfasst wurden. Wie in 0.1 beschrieben wurde für jeden testfall die Laufzeit der Maximabestimmung, der Prominenz- und Dominanzberechnung sowie der Gesamtalgorithmuslaufzeit gemessen.

tests interpretieren und bewerten. ✕ erst ganz am ende von der PA wenn sich nichts mehr ändert

Tabelle 2.1: Performance-Metriken für den Südschwarzwaldtest vor den Optimierungen

Metrik	Wert
Maximabestimmung	0.08s
Prominenzberechnung	17.71s
Dominanzberechnung	11.51s
Gesamtalgorithmuslaufzeit	29.10s

Tabelle 2.2: Performance-Metriken für den Südschwarzwaldtest nach den Optimierungen

Metrik	Wert
Maximabestimmung	0.08s
Prominenzberechnung	1.96s
Dominanzberechnung	16.85s
Gesamtalgorithmuslaufzeit	18.67s

Tabelle 2.3: Performance-Metriken für den Wetterstein vor den Optimierungen

Metrik	Wert
Maximabestimmung	0.01s
Prominenzberechnung	1.41s
Dominanzberechnung	1.92s
Gesamtalgorithmuslaufzeit	3.76s

Tabelle 2.4: Performance-Metriken für den Wetterstein nach den Optimierungen

Metrik	Wert
Maximabestimmung	0.01s
Prominenzberechnung	1.20s
Dominanzberechnung	2.54s
Gesamtalgorithmuslaufzeit	4.09s

Tabelle 2.5: Performance-Metriken für den Feldberg vor den Optimierungen

Metrik	Wert
Maximabestimmung	0.01s
Prominenzberechnung	1.20s
Dominanzberechnung	0.21s
Gesamtalgorithmuslaufzeit	1.48s

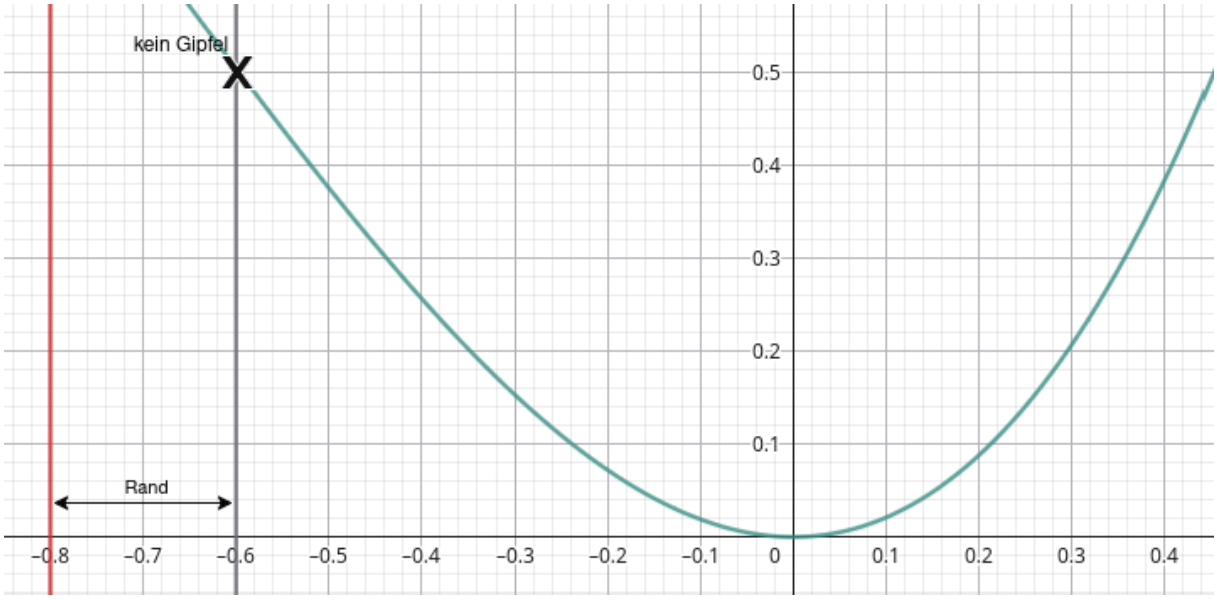

Abbildung 2.5: 2D-Geländeprofil zur Illustration der minimalen Prominenz und Dominanz bei begrenztem Kartenausschnitt.

Tabelle 2.6: Performance-Metriken für den Feldberg nach den Optimierungen

Metrik	Wert
Maximabestimmung	0.01s
Prominenzberechnung	1.24s
Dominanzberechnung	1.60s
Gesamtalgorithmuslaufzeit	2.87s

2.8 Bewertung

Die Bewertung der vorgenommenen Optimierungen des Gipfelfindungsalgorithmus erfolgt anhand von Laufzeitmessungen und der Analyse der Ergebnisse des Algorithmus. Es wurden erneut die in 1.4 beschriebenen Testfälle durchgeführt, um die Auswirkungen der Optimierungen auf die Genauigkeit der Ergebnisse zu überprüfen. Dabei lag der Fokus insbesondere auf der Prominenz- und Dominanzabweichung und der Anzahl der gefundenen realen Gipfel, sowie den fehlklassifizierten Gipfeln. Die Optimierungen haben in allen Testfällen zu einer Verbesserung der Genauigkeit des Algorithmus geführt. ein gutes Beispiel dafürr ist der Wettersteintest. Bei diesem werden nun keine nicht existierenden Gipfel mehr gefunden und fast alle realen Gipfel. Als einziger realer Gipfel wird der kleine Solstein nicht korrekt erkannt, da seine höhe in der GeoTIFF Datei neidriger angegeben

ist und er dadruch nicht mehr die Prominenzbedingung des Tests erfüllt. Außerdem haben sich die Prominenz-, sowie Dominanzabweichung stark verringert. In dem Wettersteintest ist durch die Optimierungen die Prominenzabweichung und Dominanzabweichung um ca. Faktor 4 geringer als zuvor. Bei den anderen Tests liegt die Verringerung der Abweichungen immer über dem Faktor 2 und im Mittel bei 2,75. Die Laufzeit hat sich auch verbessert. Diese wurden in 2.7 thematisiert.

3 Verarbeitung mehrerer GeoTIFF-Dateien

3.1 Motivation

Die Gipfelsuche auf umfangreichen Datensätzen erfordert eine dateiübergreifende Verarbeitung von GeoTIFF-Dateien. Diese Notwendigkeit ergibt sich aus zwei wesentlichen Gründen. Erstens begrenzt das GeoTIFF-Format die Dateigröße auf 4 GB, was für die Abbildung großer Kartenbereiche in hoher Auflösung oder ganzer Gebirge nicht ausreicht. Zweitens liegen Geodaten häufig in Form von Kacheln vor, die als separate GeoTIFF-Dateien gespeichert sind.

Um ein ganzes Gebirge zu analysieren, müssen die Daten aus diesen Kacheln entweder zu einer einzigen Datei zusammengefügt oder durch eine Logik verarbeitet werden, die eine gleichzeitige Analyse mehrerer Dateien ermöglicht. In dieser Projektarbeit wird die zweite Option verfolgt, da sie eine flexiblere und effizientere Verarbeitung ermöglicht.

Für eine laufzeiteffiziente Verarbeitung ist die Implementierung von Multithreading erforderlich. Dadurch können die Daten mehrerer Dateien parallel verarbeitet und die Ressourcen eines Mehrkernsystems optimal genutzt werden, was die zeitaufwendige Verarbeitung großer Datenmengen beschleunigt.

Neben der parallelen Verarbeitung muss auch das korrekte Zusammenführen der Daten gewährleistet sein. Die Daten aus den verschiedenen Dateien müssen so zusammengeführt werden, dass topografische Merkmale über die Dateigrenzen hinweg korrekt identifiziert werden können. Dafür muss eine Logik implementiert werden, die die Prominenz- und Dominanzberechnung über die Dateigrenzen hinweg ermöglicht, um sicherzustellen, dass die Ergebnisse der Gipfelsuche konsistent und genau sind. Die Prominenzberechnung und Dominanzberechnung auf den einzelnen Dateien wird zuvor ausgeführt, um die Datenmenge zu reduzieren, bevor die Ergebnisse über die Dateigrenzen hinweg zusammengeführt werden.

3.2 Ausführung mehrerer Dateien

In diesem Abschnitt wird die Ausführung des Algorithmus auf mehreren GeoTIFF-Dateien beschrieben. Dabei wird erläutert, wie die bestehenede Architektur verändert werden muss um eine Verarbeitung mehrerer Dateien zu ermöglichen. Dabei ist zu beachten, dass die Dateien zunächst sequenziell verabreitet werden sollen. Die parallele Verarbeitung von Dateien wird in **??** beschrieben. Außerdem soll die Möglichkeit der Verabreitung einer einzelnen Datei mit dem alten Algorithmus weiterhin bestehen bleiben. Dies ist dem geschuldet, dass manche Kartenabschnitte nur in einer Datei vorliegen.

Als ersten Schritt muss die Möglichkeit geschaffen werden, mehrere Dateien als Eingabe zu akzeptieren. Dafür muss in dem Tester aus 1 eine neue Variable namens *input_folder* erstellt werden. Diese Variable definiert den Pfad zu einem Ordner, in dem sich die GeoTIFF-Dateien befinden, die verarbeitet werden sollen. Diese Varibale darf nur gesetzt sein, wenn *input_file* nicht gesetzt ist. Außerdem muss immer eine der beiden varibalen gesetzt sein. Es muss eine Logik implementiert werden, die alle GeoTIFF-Dateien in diesem Ordner identifiziert und in einer Liste speichert. Diese Liste wird dann verwendet, um die Dateien nacheinander zu verarbeiten.

Die Verarbeitung der Dateien erfolgt in einer Schleife, die über die Liste der Dateien iteriert. Für jede Datei wird der Algorithmus wie gewohnt ausgeführt, um die Gipfel zu identifizieren. Diese Funktion gibt für jede Datei eine Liste von Gipfeln zurück, die in dieser Datei gefunden wurden. Diese Gipfel sind jedoch nur lokal für die jeweilige Datei gültig. Um zu bestimmen, in wie fern die bestimmten Gipfel über die Dateigrenzen hinweg gültig sind, wird in 3.3 beschrieben, wie die Prominenz- und Dominanzberechnung über die Dateigrenzen hinweg ermöglicht wird.

Bezüglich des Randwertproblems muss beachtet werden, dass die Gipfel, die sich im Randwert einer Datei befinden nach der aktuellen Implementierung ignoriert werden, obwohl ausreichend umgebende Daten vorhanden sind um die Prominenz- und Dominanzberechnung später dateiübergreifend auszuführen. Deshalb wird das Randwertproblem in diesem Fall nicht auf der Einzelfallebene gelöst, sondern erst bei der Zusammenführung der Ergebnisse über die Dateigrenzen hinweg. Das bedeutet, dass die Gipfel, die sich im Rand-

bereich einer Datei befinden, zunächst als gültige Gipfel betrachtet werden. Erst bei der Zusammenführung der Ergebnisse über die Dateigrenzen hinweg wird überprüft, ob diese Gipfel tatsächlich gültig sind oder ob sie aufgrund von Daten in benachbarten Dateien als ungültig betrachtet werden müssen. Diese Vorgehensweise ermöglicht es, die Gipfel, die sich im Randwert einer Datei befinden, nicht vorschnell zu verwerfen und somit potenziell gültige Gipfel zu erhalten, die sonst verloren gehen würden.

Um all diese Probleme zu lösen wird eine Klasse namens *PeakAnalyser*-Klasse erstellt, die die Agretation und das Managment der Gipfelsuche übernimmt. Dieser Klasse werden alle GeoTIFF-Dateien übergeben und sie erstellt aus diesen einzelne *SingleFileAnalyser*-Objekte, die der in 2 verwendeten *PeakAnalyser* Klasse entsprechen. Diese Klassen untersuchen nun jeweils eine Datei und geben die Ergebnisse wie oben beschreiben zurück. Dadurch können Gipfelkandidaten zunächst Resourcen effizient bestimmt werden, da jede Datei zunächst einzeln betrachtet werden kann und somit viel weniger Gipfel die aufwendigere dataeiübergreifende Analyse bewältigen müssen. Dies sorgt für eine geringere Analysezeit auf mehreren Dateien.

vereinheitlichen über Kapitel hinweg die bezeichnung der einzeldateianalyse Klasse

3.3 Zusammenführung der Ergebnisse

filemap erstellen

Dateien positionen bestimmen

globale Randgipfel ausschließen

Prominenzberechnung über die Dateigrenzen hinweg

Dominanzberechnung über die Dateigrenzen hinweg

vorschalg aus alter pa in text einarbeiten

3.4 Parallelisierung

Um nun die Gipfelsuche auf mehreren Dateien zu beschleunigen kann die Gipfelsuche auf den Einzeldateien parallelisiert werden. Das würde bedeuten, dass alle Dateien gleichzeitig untersucht werden können und die Ergebnisse der einzelnen Dateien später wieder zusammengeführt werden können. Diese Architekturveränderung kann theoretisch die Geschwindigkeit bis zu der Anzahl der Prozessorkerne erhöhen. Im Weiteren werden verschiedene Ansätze der Paralellisierung besprochen und verglichen um den für diesen Anwendungsfall besten Ansatz zu finden.

Multithreading teilt ein Programm in mehrere Threads innerhalb eines Prozesses auf. Alle Threads teilen sich den gleichen Adressraum. Dadurch ist der Datenaustausch zwischen Threads effizient, allerdings steigt das Risiko von Race Conditions bei gemeinsam genutz-

ten Datenstrukturen. Daher begrenzt der Global Interpreter Lock (GIL) in Python die gleichzeitige Ausführung von Python-Bytecode in mehreren Threads. Für gemeinsam genutzte Datenstrukturen sind zusätzlich explizite Sperren, etwa *Lock*-Objekte, erforderlich, um konsistente Schreib- und Lesezugriffe sicherzustellen.

Für die Verarbeitung mehrerer GeoTIFF-Dateien eignet sich Multithreading vor allem für I/O-lastige Teilschritte , beispielsweise das Einlesen von Dateien. Bei CPU-lastigen Berechnungen ist der Geschwindigkeitsgewinn in Python (speziell in der Standard-Implementierung CPython) durch den GIL stark limitiert. Der GIL stellt sicher, dass stets nur ein einziger Thread zur selben Zeit den Python-Bytecode ausführt. Folglich kann ein einzelner Python-Prozess selbst mit mehreren aktiven Threads immer nur maximal einen CPU-Kern physisch auslasten. Darum handelt es sich bei CPU-lastigen Aufgaben in Python um kein „echtes“, parallel ausgeführtes Multithreading über mehrere Kerne, sondern lediglich um ein schnelles, nebenläufiges Wechseln (Nebenläufigkeit) auf einem Kern. Dieser Aspekt sorgt dafür, dass der classische Multithreadingansatz in Python nicht zu einer Perfromancesteigerung führt. Daher muss bei dem Gipfelsuchalgorithmus auf andere Verfahren zurückgegriffen werden.

link zu ui reaktivität

Als Alternative kann Multiprocessing eingesetzt werden. Dabei wird jede Datei in einem separat gestarteten Prozess analysiert, von denen jeder seinen eigenen Interpreter samt eigenem GIL besitzt. Nur dieser Ansatz erlaubt in Python eine echte Parallelität auf mehreren Prozessorkernen, erfordert jedoch mehr Speicher und einen höheren Verwaltungsaufwand für die Prozesskommunikation. Dieser Ansatz hat einen entscheidenen Nachteil, denn er sorgt für eine Hohe Ramauslastung. Dies hängt damit zusammen, dass Python in jedem Prozess alle Daten halten muss die in diesem Prozess verwendet werden. Diese Tatsache sorgt bei der parallelen Ausführung mehrerer Dateien (nach 3.2) für den doppelten Ramverbrauch im Gegensatz zur sequenzieler Ausführung. Bei der Zusammenführung (nach 3.3) sorgt das für eine unmögliche Ausführung des Algorihtmus. Das liegt daran, dass hier für jeden Gipfel, der untersucht werden muss alle GeoTIFF-Dateien gespeichert werden müssen. Das bedeutet der Ramverbrauch ist bei der Zusammenführung um ein vielfaches höher als zuvor. Dies führt in den meisten fällen zu einem Absturz des Computers.

Um Multiprocessing nun rameffizienter umzusetzen gibt es drei Möglichkeiten. Die erste wäre eine Gruppierung der Gipfel. Dies meint, dass die Gipfel oder Dateien vor der Untersuchtung zuerst in Gruppen aufgeteilt werden und danach für jede der Gruppen ein Prozess erstellt wird, der die Analyse für diese ausführt. Wenn die Anzahl der Gruppen immer dem *max(Prozessorkernen, Analyseobjekten)* entspricht wird durch diese Änderung die Geschwindigkeit des Multiprocessing nahezu beibehalten. Er ist lediglich wegen dem Austausch der Analysepbjekte nahc jedem Durchlauf langsamer. Der Ramverbrauch ist außerdem geringer. Er ist je nach Gerät und GeoTIFF-Dateigröße immernoch im kritischen Bereich und kann zum Einfrieren oder Abstürzen des Computer führen.

Eine weitere Möglichkeit wäre es anstatt bei der Erstellung eines prozesses alle Daten zu übergeben, jedem Prozess immer nur die Daten zu geben, wenn er diese benötigt. Dies würde dazu führen, dass dauerhaft weniger Speicher beansprucht wird, würde aber die Ausführzeit erhöhen. Zudem wäre dies sehr aufwändig zu implementieren. Alternativ hier zu können die Daten alle anstatt an den Prozess selbst übergeben zu werden in einem gemeinsamen Speicher (shared Memory) liegen, auf den der jeder Prozess selbst zugreift. Dies würde die Ausführgeschwindigkeit des Algorithmus zwar minimal verringern, aber den Ramverbrauch enorm senken.

Für den vorliegenden Anwendungsfall ist daher eine differenzierte Strategie zweckmäßig: I/O-lastige Schritte werden mittels Multithreading parallelisiert, CPU-lastige Analysen pro Datei mittels Multiprocessing mit shared Memory. Die Ergebnisse der Einzelanalysen werden anschließend zentral zusammengeführt und in 3.3 dateiübergreifend validiert. Diese validierung wird auch mit der Hilfe von Multiprozessing mit shared Memory parallelisiert.

Bewertung des neuen Allgorithmus

4 Reengineering der Benutzeroberfläche

intro zu 7

4.1 Motivation

Die bestehende Benutzeroberfläche (User Interface, UI) dient als Schnittstelle für den ursprünglichen Algorithmus. Sie weist funktionale und strukturelle Defizite auf und ist nicht auf den optimierten Algorithmus ausgelegt. Die identifizierten Probleme umfassen mangelhafte Reaktivität, eingeschränkte Navigation sowie architektonische Designmängel.

Abkürzung UI in acronyms.tex

Ein wesentliches Defizit ist die fehlende Reaktivität. Während der Ausführung des Algorithmus friert die Benutzeroberfläche ein, wodurch jegliche Interaktion während der Rechenzeit blockiert wird. Zudem ist die Skalierung der Ansichtskomponenten statisch. Die grafische Kartenansicht und die Ergebnistabelle besitzen ein festes Größenverhältnis und lassen sich nicht variabel vom Nutzer anpassen. Das sorgt dafür, dass dem Nutzer keine detailierte Sicht auf beide Anzeigen verwäht bleibt. Die Übersichtlichkeit der Applikation wird durch eine fragmentierte Menüführung beeinträchtigt. Konfigurationsparameter verteilen sich inkonsistent auf eine seitliche Menüleiste und auf separate Untermenüs. Darüber hinaus ist die visuelle Zuordnebarkeit der Ergebnisse eingeschränkt. Gipfelkoordinaten aus der Tabelle müssen manuell auf der Karte lokalisiert werden. Eine interaktive Verknüpfung, wie etwa die Hervorhebung eines Gipfels auf der Karte durch die Selektion des zugehörigen Tabelleneintrags, fehlt.

Auf architektonischer Ebene bestehen Mängel in der Codequalität. Die Trennung von Benutzeroberfläche und Geschäftslogik ist unzureichend umgesetzt. Algorithmus-spezifische Verarbeitungsschritte werden direkt in den UI-Komponenten ausgeführt. Obwohl ein objektorientiertes Framework als technologische Basis dient, folgt die Implementierung nicht konsequent objektorientierten Prinzipien. Diese Faktoren mindern die Wartbarkeit sowie die Erweiterbarkeit des Systems. In Summe motivieren diese Defizite ein vollständiges Reengineering der Applikationsoberfläche.

4.2 Umsetzung

4.2.1 Fortschritt-Visualisierung

Um dem Nutzer Rückmeldung zu geben, wie weit der Algorithmus mit der Gipfelsuche fortgeschritten ist wird auf zwei seperate mechanismen gesetzt. Zum einen wird auf einen Ladebalken, wie er Beispielsweise im Windows Update Bildschirm zu sehen ist gesetzt. Zusätzlich werden ausführliche Logs in der Kommandozeile implementiert, die zudem auch für die Debugging Zwecke verwendet werden können. Die Kommandozeilenausgabe ist leit mit der Hilfe der Python Bibliothek logging zu implementieren. Dem Ladebalken hingegen muss immer mitgeteilt werden wie weit die Gipfel Analyse Fortgeschritten ist. Dies ist besonders schwierig, da zu Beginn die genaue Länge nicht abschätzbar ist und auch auf gleich großen Dateien und mit gleiochen eingabeparametern stark variiert. Daher wird eine Klasse - Analysis State - implementiert, die von dem Algorithmus REückmeldung erhält, wie weit die Analyse fortgeschritten ist und dies in eine Prozentzahl übersetzt. Diese Übersetzung muss für Mehrdateiausführungen anders erfolgen wie für Einzeldateien.

Im Grunde nutzt der Analysis State die Tatsache aus, dass ein Schritt immer die Anzahl der zu untersuchenden Erhebungen für den Folgeschritt bestimmt. Das bedeutet, dass die für die Prominenzberechnung zu überprüfenden Gipfel von der Lokalen Maxima Bestimmung entdeckt gefunden werden. Da vor der lokalen Maximabestimmung kein Schritt existiert, kann jedoch hier keine Aussage über die zu erwartenden Werte getroffene werden. Das stell kein größeres Problem da, da dieser Schritt in nahezu jedem Fall der schnellste Schritt ist und alle Werte nicht nacheinander sondern gleichzeitig bestimmt. Meist ist dieser Schritt ohnehin so schnell, dass der Nutzer es diesen Schritt nicht bemerkt. Um nun pro Schritt sagen zu können, wie weit der Algorithmus mit den Berechnung ist, wird die Anzahl der bearbeiteten Elemente durch die Anzahl der zu bearbeitenden Elemente geteilt. Diese Zahl gibt nun an, wie viele Prozent der zu bearbeitenden Elemente bereits bearbeit sind. Wenn nun angenommen wird, dass jedes Element gleich viel Zeit für die Verarbeitung benötigt, kann dieser Prozentwert auch als zeitliche Einschätzung verwendet werden. Da in Realität nicht jedes Element genau die gleiche Zeit benötigt wird diese Art der Fortschittbestimmung nie genau sein. Es stellt aber einen guten und

verständlichen Messwert da und ist daher für diese Applikation ausreichend. Um über die Schritte einen Gesammtfortschritt zu bestimmen müssen die einzelnen Teilfortschritte gewichtet zusammenaddiert werden. Die Gewichte dafür wurden empirisch bestimmt mit der Hilfen von Ausführungen verscheidengroßer und komplexer Dateien bestimmt.

rechtschreibung hier?

Wenn mehrere Dateien untersucht werden, werden viele Schritte parallel ausgeführt. Das bedeutet, dass die bisherige Methode zur Bestimmung des Status nicht mehr funktioniert. Diese Tatsache rührt daher, dass um abzuschätzen wie weit der Algorithmus vorrangeschritten ist, die zu untersuchenden Gipfel des vorherigen Schrittes schon im gesammten errechnet worden sein müssen. Es ist jedoch für einen großteil der Ausführzeit noch nicht die Gesammtanzahl der zu untersuchenden Gipfel bekannt. Daher wird sich mit einer Hochrechnung beholfen. Diese Hochrechnung schaut pro Schritt wie viele Gipfel bereits bekannt aus wievielen Dateien bekannt sind und rechnet diese Zahl auf eine zu erwartende Menge für alle Dateien hoch. Das bedeutet, dass wenn der Algorithmus in 10 Dateien 50 Gipfel in der Prominenzberechnung findet, wird angenommen, dass auf 20 Dateien (allen zu untersuchenden Dateien in diesem Beispiel) 100 Gipfel gefunden werden. Diese vorgehensweise bietet jedoch auch ein Problem. Es ist möglich das durch sehr ungleiche Daten der Fortschritt hin und her springt oder für eine Zeit rückwärtslaufend erscheint. In unserem Beipiel würde dass auftretten, wenn in der elften untersuchten Datei 50 Gipfel zu finden ist. Dann würde die errechnete Abschätzung der Gesamtanzahl der Gipfel auf 181 steigen. Dass würde den prozentualen Fortschritt fallen lassen, wenn in der zwischenzeit in der Dominanzbestimmung nicht genügend Gipfel untersucht worden würden. Dieser wurde in Tests in vernachläsigbarer Ausprägung nachgewiesen.

- mehrfachdateien

- multi threading kontextwechsel langsam - daten je Prozess - auf Multithreading ändern - Fortschrittanzeige die Gewichte besser amchen oder dynamischer - Ausblick der raster search Algorithmus für Dominanz